

Question 1:- Merge Sort algorithm (Easy):-

Input:-

- First line contains an integer N , representing the no. of placed students.
- Second line contains N space-separated integers, representing package values.

Output:- print the package values in ascending order.

⇒ Approach:-

- ~~Sort~~ Divide the array into two halves.
- Keep dividing until each subarray contains only one element.
- Merge the subarrays in sorted order.
- Continue merging until the complete array becomes sorted.

* Code:-

```
Syntax:- void merge(vector<int> &arr, int low, int mid,
                int high) {
    vector<int> ans;
    int i = low;
    int j = mid + 1;
    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            ans.push_back(arr[i]);
            i++;
        } else {
            ans.push_back(arr[j]);
            j++;
        }
    }
}
```

```

while (i <= mid) {
    ans.push_back(arr[i]);
    i++;
}
while (j <= high) {
    ans.push_back(arr[j]);
    j++;
}
for (int k = low; k <= high; k++) {
    arr[k] = ans[k - low];
}

void mergeSort(vector<int> &arr, int low, int high) {
    if (low >= high)
        return;
    int mid = (low + high) / 2;
    mergeSort(arr, low, mid);
    mergeSort(arr, mid + 1, high);
    merge(arr, low, mid, high);
}

```

* Time Complexity :- ~~$O(n^2)$~~ $O(n \log n)$

Question - 2 :- Merge Sort (medium)

* Input :-

- First line contains integer N .
- Second line contains N space-separated processing times.

* Output :-

- Sorted processing times.
- Median processing time.
- Count of orders greater than median.
- Diff b/w max. and min. processing time.

⇒ Approach :-

- Read the input array.
- Apply merge sort to sort the array.
- Print the sorted array.
- Find the median.
- Count elements greater than the median.
- Calculate $\text{max.} - \text{min.}$

Code:-

```
Syntax:- int main() {  
    int n;  
    cin >> n;  
    vector<int> arr(n);  
    for(int i=0; i<n; i++){  
        cin >> arr[i];  
    }  
    mergesort(arr, 0, n-1);  
    for cout << "Sorted Array: ";  
    for(int i=0; i<n; i++){  
        cout << arr[i];  
    }  
    cout << endl;  
    int median = arr[n/2];  
    cout << "Median: " << median;  
    int count = 0;  
    for(int i=0; i<n; i++){  
        if(arr[i] > median)  
            count++;  
    }  
    cout << count << endl;  
    int diff = arr[n-1] - arr[0];  
    cout << diff;  
    return 0;  
}
```

Question - 3 :- Quick Sort (Easy)

Input :-

- First line contains Integer N .
- Second ~~line~~ line contains N space-separated response time.

Output :-

- Print the response times in ascending order.

* Algorithm :-

- Choose the last element as the pivot.
- Rearrange the array so that:
 - Elements smaller than the pivot come to the left.
 - Elements greater than the pivot come to the right.
- Place the pivot in its correct position.
- Repeat the same process for the left and right subarrays.

* Code :-

Syntax:-

```
int partition(vector<int> &arr, int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;  
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
    swap(arr[i + 1], arr[high]);  
    return i + 1;  
}  
  
void quickSort(vector<int> &arr, int low, int high) {  
    if (low < high) {  
        int pivotIndex = partition(arr, low, high);  
        quickSort(arr, low, pivotIndex - 1);  
        quickSort(arr, pivotIndex + 1, high);  
    }  
}
```


Question 4 - ~~Max~~ Quick Sort (medium)

Input-

- First line contains Integer N .
- Second line contains N space-separated trade values.

Output-

- Sorted trade values (descending).
- Top 5 values.
- Average of Top 5 values.
- Count of values greater than overall average.

⇒ Algorithm:-

- Read the array.
- Apply Quick Sort (Descending order)
- Print the sorted array.
- Print the ~~first~~ first 5 element.
- Calculate the average of the Top 5.
- Calculate the Overall average.
- Count elements greater than the overall average.

★ Code:-

Syntax:-

```
int main() {
    int n;
    cin >> n;
    vector<long long> arr(n);
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    quickSort(arr, 0, n-1);
    cout << "Sorted array";
    for (int i = 0; i < n; i++) {
        cout << arr[i];
    }
    cout << endl;
    cout << "Top 5:";
    long long sumTop = 0;
```

```
for (int i=0; i<5; i++) {
```

```
    cout << arr[i] << " ";
```

```
    sumTop += arr[i];
```

```
} cout << endl;
```

```
double topAverage = sumTop/5.0;
```

```
cout << topAverage;
```

```
long long total = 0;
```

```
for (int i=0; i<n; i++)
```

```
    total += arr[i];
```

```
double overallAverage = total * 1.0/n;
```

```
int count = 0;
```

```
for (int i=0; i<n; i++) {
```

```
    if (arr[i] > overallAverage)
```

```
        count++;
```

```
}
```

```
cout << count;
```

```
return 0;
```

```
}
```

Question 5:- HeapSort (Easy) -

★ Algorithm :-

- Build a max heap from the given array.
- Swap the root (largest element) with the last element.
- Reduce the heap size by one.
- Apply Heapify on the root.
- Repeat until only one element remains.

Code -

Syntax:-

```
void heapify(vector<int> &arr, int n, int i){
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if(left < n && arr[left] > arr[largest])
        largest = left;
    if(right < n && arr[right] > arr[largest])
        largest = right;
    if(largest != i){
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
```

```
void heapsort(vector<int> &arr, int n){
    for(int i = n/2 - 1; i >= 0; i--){
        heapify(arr, n, i);
    }
    for(int i = n - 1; i > 0; i--){
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

Question - 6:- Heapsort (Medium)

⇒ Algorithm:-

- Read the input array.
- Build a Max Heap.
- Perform Heapsort.
- Print the sorted array.
- Find the min. and max values.
- Calculate the average.
- Count the values smaller than the average.
- Calculate the percentage.

Code:-

Syntax:-

```
int main() {  
    int n;  
    cin >> n;  
    vector<int> arr(n);  
    for (int i=0; i<n; i++)  
        cin >> arr[i];  
    heapSort(arr, n);  
    cout << "Sorted Array: ";  
    for (int i=0; i<n; i++)  
        cout << arr[i];  
    cout << endl;  
  
    cout << arr[0];  
    cout << arr[n-1];  
  
    double sum = 0;  
    for (int i=0; i<n; i++)  
        sum += arr[i];  
    double average = sum/n;  
    cout << fixed << setprecision(2);  
    cout << average;  
    int count = 0;  
    for (int i=0; i<n; i++) {  
        if (arr[i] < average)  
            count++;  
    }  
    cout << count;  
    double Percentage = (count * 100.0) / n;  
    cout << percentage;  
    return 0;  
}
```